

Semaphores

Part I

Sistemas de Computadores
Departamento de Engenharia Informática
Instituto Superior de Engenharia do Porto

Luís Nogueira (lmn@isep.ipp.pt)

Introduction

- Multiple processes often need to share resources (e.g., memory, files, devices)
- Without proper coordination, data corruption, inconsistency, and unpredictable behavior can occur
- We will explore increasingly complex synchronization challenges, starting with solving a **race condition** before moving to more advanced topics
 - Lectures offer a comprehensive and in-depth discussion on the topic

Race condition

- Two or more processes attempt to access a shared resource (e.g., memory, file, or device), with at least one performing a write
- A race condition arises when the system's correctness depends on the **unpredictable order of execution** of these processes
- Uncontrolled race conditions can lead to data corruption, inconsistencies, or unexpected system behavior

Race condition

- What is the expected final value of shm_counter?

```
*shm_counter = 0;      /* initial value */

pid = fork();

if (pid > 0){
    for (i=0; i<100; i++)
        (*shm_counter)++;
    aux = wait(&status);
    printf("Counter: %d\n", *shm_counter);
}
else {
    for (i=0; i<100; i++)
        (*shm_counter)++;
}
```

Race condition

- What is the expected final value of `shm_counter`?

One integer, in shared memory

```
*shm_counter = 0;    /* initial value */

pid = fork();

if (pid > 0) {
    for (i=0; i<100; i++)
        (*shm_counter)++;
    aux = wait(&status);
    printf("Counter: %d\n", *shm_counter);
}
else {
    for (i=0; i<100; i++)
        (*shm_counter)++;
}
```

movl (%rsi), %edx
incl %edx
movl %edx, (%rsi)

movl (%rsi), %ecx
incl %ecx
movl %ecx, (%rsi)

Race condition

- What is the expected final value of `shm_counter`?

```
*shm_counter = 0;      /* initial value */

pid = fork();

if (pid > 0) {
    for (i=0; i<100; i++)
        (*shm_counter)++;
    aux = wait(&status);
    printf("Counter: %d\n", *shm_counter);
}
else {
    for (i=0; i<100; i++)
        (*shm_counter)++;
}
```

Unpredictable order of execution may lead to inconsistent results

`*shm_counter = 0`

`movl (%rdi), %edx`

Parent

`movl (%rdi), %ecx`

`incl %ecx`

`movl %ecx, (%rdi)`

Child

`incl %edx`

`movl %edx, (%rdi)`

Parent

`*shm_counter = 1 // should be 2`

Synchronization mechanisms

- To ensure data consistency, it is essential to provide mechanisms that enforce a specific order between cooperating processes
- Semaphores are a classic synchronization mechanism that can be used to:
 1. Limit the simultaneous access of n processes to a shared resource
 2. Signal the occurrence of events

Semaphore

- **Conceptually, semaphores are integer counters**
 - In POSIX implementations, their value is always kept ≥ 0
- **Attempting to decrement the semaphore below zero blocks the process until the operation becomes possible**
 - i.e., the semaphore remains non-negative
 - In this case, the process is placed in the semaphore's **waiting queue and releases the CPU**

Semaphore

- A semaphore can only be **manipulated through a specific interface**
 - Traditionally, those operations are called `down()` and `up()`
 - In the POSIX interface, these operations are called `sem_wait()` and `sem_post()`, respectively
- The OS must guarantee that these operations are **atomic**
 - The operation appears indivisible — no other process can observe a partial execution

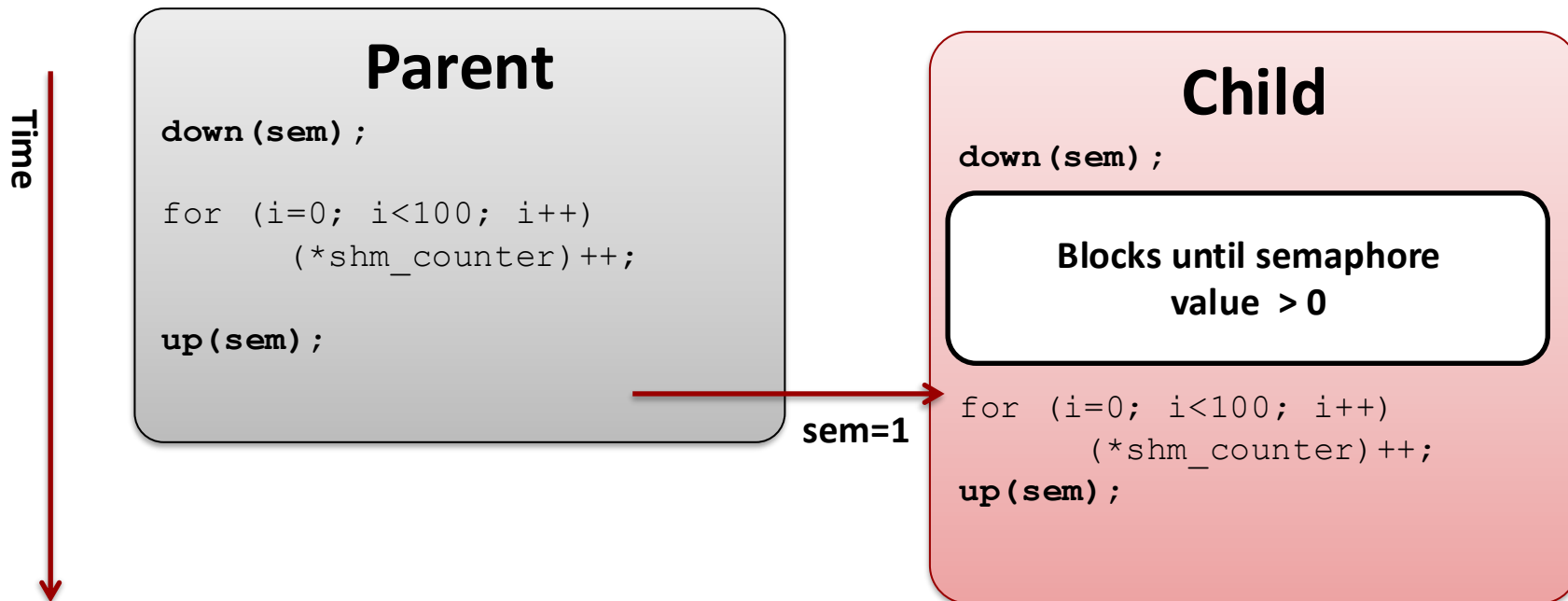
Initial value

- The initial value of a semaphore plays a crucial role in its behavior
- 1 → Binary semaphore (mutex-like behaviour)
- 0 → Event synchronization (wait-for-signal)
- ≥ 0 → Counting semaphore (N concurrent accesses allowed)

Example – Mutual exclusion

■ Based in previous example (slide 3)

- Initial value of semaphore is 1
- Assume parent executes `down()` first
 - The correctness does NOT depend on which process executes first



Advantages

■ Efficiency

- Can be efficiently implemented across different platforms
- Avoids busy waiting by blocking the process in the kernel

■ Flexibility

- Limits resource access to n processes
- Signals events, allowing solutions to a wide range of synchronization problems

Challenges

■ Prone to synchronization errors

- Failing to properly protect shared resources can cause inconsistencies or even deadlocks (where processes remain blocked indefinitely)

■ Unpredictable execution order

- When multiple processes wait for a resource, the order in which they are served may be unpredictable
- POSIX does not guarantee FIFO ordering of waiting processes

POSIX named semaphores in Linux

■ `sem_open()`

- Create/open a semaphore

■ `sem_post()`

- Increment (up) the semaphore's value

■ `sem_wait()`, `sem_trywait()`, `sem_timedwait()`

- Try to decrement (down) the semaphore's value

■ `sem_getvalue()`

- Get the current semaphore's value

POSIX named semaphores in Linux

■ **sem_close()**

- Releases the process's reference to the semaphore

■ **sem_unlink()**

- Removes the semaphore name from the system
- The semaphore is destroyed only after it has been unlinked and all processes have closed it

The `sem_open` function

```
#include <semaphore.h>

sem_t *sem_open(const char *name, int oflag, mode_t mode,
                unsigned int value);
sem_t *sem_open(const char *name, int oflag);
```

■ Creates a new or opens an existing semaphore

- The variant with two parameters is used to open an existent semaphore

■ Returns

- A pointer to `sem_t`, on success
- `SEM_FAILED`, on error

The `sem_open` function

```
#include <semaphore.h>

sem_t *sem_open(const char *name, int oflag, mode_t mode,
                unsigned int value);
```

- The semaphore is identified by name
- The `oflag` argument specifies flags that control the operation of the call
 - `O_CREAT`: creates a new semaphore, if it not exists
 - `O_EXCL`: (with `O_CREAT`) creates a new semaphore if not exists, error if it already exists

The `sem_open` function

```
#include <semaphore.h>

sem_t *sem_open(const char *name, int oflag, mode_t mode,
                unsigned int value);
```

- **The `mode` parameter specifies the permissions to be placed on the new semaphore**
 - Both read and write permission should be granted to each process that will access the semaphore
- **The `value` parameter specifies the initial value for the new semaphore**

The `sem_post` function

```
#include <semaphore.h>

int sem_post(sem_t *sem);
```

- Increases the semaphore value by one unit
- `sem` is the semaphore's address returned by `sem_open()`
- Returns:
 - 0, on success
 - -1, on failure

The `sem_wait` function

```
#include <semaphore.h>

int sem_wait(sem_t *sem);
```

- **Attempts to decrement the semaphore value by one unit**
 - If the semaphore value is greater than zero, decrements immediately
 - Otherwise, blocks until the decrement becomes possible
- **`sem` is the semaphore's address returned by `sem_open()`**
- **Returns:**
 - 0, on success
 - -1, on failure

`sem_trywait` and `sem_timedwait`

```
int sem_trywait(sem_t *sem);  
  
int sem_timedwait(sem_t *sem, const struct timespec *timeout);
```

- The `sem_trywait()` function is similar to `sem_wait()` but **does not block**
 - If the semaphore value is greater than zero, decrements immediately.
 - Otherwise, returns an error (-1)
- The `sem_timedwait()` function is similar to `sem_wait()` but **blocks only until a specified timeout**
 - If the decrement succeeds before the timeout, it proceeds normally
 - If the timeout expires before the semaphore becomes available, it returns an error (-1)

The `sem_getvalue` function

```
#include <semaphore.h>

int sem_getvalue(sem_t * sem, int * sval);
```

■ Gets the current semaphore value

- The value of the semaphore **may already have changed** by the time `sem_getvalue()` returns
- Should not be used for synchronization decisions!

■ `sem` is the semaphore's address returned by `sem_open()` and `sval` is the address where the value should be returned

■ Returns:

- 0, on success
- -1, on failure

The `sem_close` function

```
#include <semaphore.h>

int sem_close(sem_t * sem);
```

- **Releases the process's reference to the semaphore**
 - Does not remove it from the system
 - Similar to `close()` for file descriptors
- **`sem` is the semaphore's address returned by `sem_open()`**
- **Returns:**
 - 0, on success
 - -1, on failure

The `sem_unlink` function

```
#include <semaphore.h>

int sem_unlink(const char* name);
```

■ Marks the semaphore for removal

- The semaphore is physically deleted once all processes that have it open (i.e., are actively using it) close their connections

■ `name` is the semaphore's name (**not** the address)

■ Returns:

- 0, on success
- -1, on failure

Example – mutual exclusion

```
int main(){
    int *shm_counter = NULL;          /* pointer to shared memory object */
    sem_t *sem;

    /* create, open and map shared memory */
    ...

    /* create semaphore with initial value of 1 */
    /* this will fail if it already exists (O_CREAT | O_EXCL) */
    if((sem = sem_open("/sem_excl", O_CREAT|O_EXCL, 0644, 1)) ==
SEM_FAILED){
        perror("sem_open");
        exit(1);
    }

    *shm_counter = 0;                 /* initial counter's value */
}
```

Example – mutual exclusion

```
pid = fork();

if (pid == 0){
    sem_wait(sem);
    for (i=0; i<500000; i++)
        (*shm_counter)++;
    sem_post(sem);
    exit(0);
}else{
    sem_wait(sem);
    for (i=0; i<500000; i++)
        (*shm_counter)++;
    sem_post(sem);
}

wait(NULL);
printf("Counter: %d\n", *shm_counter);

/* closes and removes semaphore */
sem_close(sem);
sem_unlink("/sem_excl");

return 0;
}
```

Exercise 1

- **Create a shared memory area between two processes to store two integers**
 - The parent process will write values, and the child process will read and print them
- **Use semaphores for synchronization, avoiding active waiting (e.g., busy waiting or polling)**
 - Ensure that the child process waits for the parent to finish writing before accessing the shared memory
- **The parent process waits for the child process to finish and then removes the shared memory segment and the semaphore**

Exercise 2

- **Extend the previous exercise to allow the parent process to write 3 sets of 2 integers into the shared memory**
 - Each new set should only be written when the child process has confirmed that it has read the previous set
- **The parent waits for the child to finish and then removes the shared memory segment and the semaphores**